

Jellyscope Calculation and Display Reference

Jellyscope

October 2, 2026

Abstract

This document defines the current Jellyscope calculations, units, formatting, precision, and invalid-value behavior. It covers the clump inspector, WCS, separations, image stretches, RGB composites, and static data flow.

1 Data flow and precision

The bake CLI reads a primary-HDU FITS cube as f64 values and reads clump metadata from `clumps_properties.csv` and pixel assignments from `clumps_pixels.csv`. It writes one little-endian f32 plane per filter, an i32 pixel-to-clump grid, and `manifest.json`. The f32 planes are only a transport representation: the WASM viewer widens them back to f64 before running display mathematics. RGBA bytes are produced in the viewer after a stretch or composite; final display channels are 8-bit.

2 Clump inspector fields

Catalog values are copied into the manifest and are not recomputed from area, pixels, or other SED fields. Optional missing values serialize as JSON `null` and display as an em dash.

Displayed row	Manifest field	Meaning and formatting
Clump ID	<code>id</code>	Catalog <code>clump_id</code> ; integer.
Component	<code>component</code>	Catalog component name, capitalized.
Inside disk	<code>inside</code>	Optional Boolean: Yes, No, or em dash.
Area (pixels)	<code>area_pix</code>	Catalog integer; not recomputed from the grid.
Area (arcsec ²)	<code>area_arcsec2</code>	Catalog value; four decimal places.
Area (kpc ²)	<code>area_kpc2</code>	Catalog value; four decimal places.
R_{eff} (arcsec)	<code>r_eff_arcsec</code>	Optional catalog value; four decimal places.
R_{eff} (kpc)	<code>r_eff_kpc</code>	Optional catalog value; four decimal places.
Centroid x	<code>x0</code>	Catalog image coordinate; one decimal place.
Centroid y	<code>y0</code>	Catalog image coordinate; one decimal place.
RA (deg)	<code>ra_deg</code>	Derived from the centroid and affine WCS; six decimal places.
Dec (deg)	<code>dec_deg</code>	Derived from the centroid and affine WCS; six decimal places.
$\log_{10}(M_{\star}/M_{\odot})$	<code>mass</code>	Linear catalog mass in $M_{\odot}$ transformed with $\log_{10}$; three decimals.
SFR ($M_{\odot}/\text{yr}$)	<code>sfr_avg</code>	Optional catalog value; four decimal places.
sSFR (yr^{-1})	<code>ssfr_avg</code>	Optional catalog value; scientific notation with three decimals.

Displayed row	Manifest field	Meaning and formatting
$\log Z/Z_\odot$	<code>logzsol</code>	Optional catalog value; three decimal places.
Dust τ_2	<code>dust2</code>	Optional catalog value; three decimal places.
Age (Gyr)	<code>tage</code>	Optional catalog value; three decimal places.
$\log U$ (gas)	<code>gas_logu</code>	Optional catalog value; two decimal places.

The precise stellar-mass calculation is

$$m_{\text{display}} = \log_{10} \left(\frac{M_\star}{M_\odot} \right). \quad (1)$$

The catalog field `mass` is the positive linear number of solar masses. For example, 3,007,335.42439138 becomes 6.478. A missing or non-positive mass is unavailable rather than a valid logarithm. The UI retains the existing Python-compatible label $\log M_\star (M_\odot)$.

The parser requires `clump_id`, `area_pix`, `area_arcsec2`, `x0`, `y0`, `area_kpc2`, and `component`. The SED fields, effective radii, and `inside` are optional.

3 Centroid to sky coordinates

FITS reference pixels are one-based. For the rotation-free, axis-aligned WCS, Jellyscope sets

$$p_x = \text{CRPIX1} - 1, \quad p_y = \text{CRPIX2} - 1, \quad (2)$$

$$s_x = \text{PC1_1 CDELTA1}, \quad s_y = \text{PC2_2 CDELTA2}, \quad (3)$$

$$c = \cos(\text{CRVAL2} \pi / 180). \quad (4)$$

For centroid (x_0, y_0) it computes

$$\text{RA} = \text{CRVAL1} + \frac{s_x(x_0 - p_x)}{c}, \quad (5)$$

$$\text{Dec} = \text{CRVAL2} + s_y(y_0 - p_y). \quad (6)$$

Only browser display rounding is applied afterward. The approximation assumes no rotation, skew, CD, or SIP distortion. The manifest pixel scale is

$$p_{\text{arcsec}} = 3600 \frac{|s_x| + |s_y|}{2}. \quad (7)$$

4 Pairwise angular separation

The browser converts manifest RA/Dec to radians and uses

$$a = \sin^2 \left(\frac{\delta_2 - \delta_1}{2} \right) + \cos(\delta_1) \cos(\delta_2) \sin^2 \left(\frac{\alpha_2 - \alpha_1}{2} \right), \quad (8)$$

$$\theta = 2 \arcsin(\sqrt{a}), \quad (9)$$

$$d_{\text{arcsec}} = \theta \frac{180}{\pi} 3600. \quad (10)$$

Selected clumps are sorted by ID; the lower triangle is filled and formatted to two decimals, the upper triangle is empty, and the diagonal is an em dash.

5 Single-filter stretches

For the log stretch, finite positive fluxes define the 10th and 99.98th percentiles v_{10} and $v_{99.98}$. The viewer evaluates

$$x_c = \text{clip}(x, v_{10}, v_{99.98}), \quad (11)$$

$$u = \frac{x_c - v_{10}}{v_{99.98} - v_{10} + 10^{-10}}, \quad (12)$$

$$L(x) = \frac{\ln(200u + 1)}{\ln(201)}. \quad (13)$$

Non-positive values clip to zero; NaN values remain invalid and transparent.

For the Lupton-asinh stretch, finite pixels are sigma-clipped around their median at $\pm 3\sigma$ for at most five iterations. With resulting background median m , population standard deviation σ , $Q = 8$, and $\alpha = 0.02/(\sigma + 10^{-10})$,

$$A(x) = \frac{\text{asinh}(\alpha Q(x - m))}{Q}. \quad (14)$$

The result is divided by the 99.5th percentile of finite $A(x)$ and clipped to $[0, 1]$; NaN input remains transparent.

6 RGB composites

The default percentile-asinh recipe processes each band independently. It subtracts the finite median, clips to the 10th–99.9th percentile range, maps with

$$y = \frac{\text{asinh}(u/0.1)}{\text{asinh}(10)}, \quad (15)$$

sets values below 0.05 to zero, applies weights (1.0, 1.02, 1.02), clips to $[0, 1]$, and converts to bytes. A pixel invalid in any band becomes black.

The Lupton recipe uses $I = (R + G + B)/3$, the same background estimate, and a user-selected Q (default 8):

$$f(I) = \frac{\text{asinh}(\alpha Q(I - m))}{Q}, \quad r = \frac{f(I)}{I - m} \quad \text{when } I - m > 0. \quad (16)$$

Each background-subtracted channel is multiplied by r , clamped at zero, and renormalized if a channel exceeds one. Positive channels are then globally normalized by their 99.5th percentile, clipped to $[0, 1]$, and converted to 8-bit channels.

7 Pixel grid and boundaries

The i32 grid is row-major with index $yN_x + x$; empty pixels contain -1 . Out-of-bounds assignments are discarded. Rust builds each boundary with Andrew’s monotone-chain convex hull and closes the polygon. The browser applies two Chaikin corner-cutting iterations for the overlay only; catalog area and pixel membership do not change.

8 Implementation references

- `crates/bake/src/clumps.rs`: catalog parsing, grid, and hull;
- `crates/bake/src/wcs.rs`: affine WCS and separation golden;
- `crates/bake/src/main.rs`: FITS-to-manifest assembly and f32 encoding;
- `crates/core/src/stretch.rs`: single-filter stretches;
- `crates/core/src/composite.rs`: RGB recipes;
- `web/glue.js`: property formatting, readout, separations, and tooltips.